Test Plan 2 — Product Public Sanity (No-Login Dev Calculator)

Project	`product`
Specs	`tests/product/api/public-app-sanity.spec.ts`, `tests/product/api/token-sanity.spec.ts`
Default filter	none (all specs in `product` run)
Target	Dev calculator — `QA_TARGET_ENV` (default **dev** `dev1.app.organize.organuz.com`)
Page object	`tests/product/support/ProductAppPage.ts`
Cases	8 (5 public-app + 3 token)
Skips	Only on a genuine dev outage (gateway/gate down, no UI token)
Skill	`product-public-sanity`

Scope

Public, no-login checks against the live dev product calculator. The tests open the app through the shared password gate (`PRODUCT_PLATFORM_PASSWORD`), confirm the calculator shell loads for a signed-out visitor, and verify the front-end's backend token. "Token" here means the credential the UI sends with its backend calls; the tests check it is well-formed and transmitted securely — in the request body, over HTTPS, and never in the URL. There is no sign-in and no personal data.

Preconditions

- The live dev app is reachable, and `PRODUCT_PLATFORM_PASSWORD` is set (in the local `.env` file or as a CI secret).
- `TokenInterceptor` (a helper that watches the UI's outgoing backend calls) can observe those calls in order to extract the token.

Gating (sanctioned skip)

- **AppUnavailableError / OtpUnavailableError** → the whole group skips, with a clear reason: "dev app unavailable (not a product bug)".
- **token-sanity** also skips when no UI token can be extracted at all. But if a token IS observed and comes back malformed, or drifted from what config expects, the test **fails** — that is a real regression.

Cases

`public-app-sanity.spec.ts` — "Public dev calculator sanity" (`@product @sanity`)

ID	CASE	ASSERTS
PUB-01	The dev calculator shell loads after unlocking the password gate	App shell renders once the gate is unlocked
PUB-02	A fresh visitor is signed out on the public calculator	No authenticated session for a first-time visitor
PUB-03	The login entry point is available to a signed-out visitor	A visible path to sign in exists
PUB-04	The app is served over HTTPS on the configured dev host	Origin is HTTPS on the expected dev host
PUB-05	The calculator issues a public backend call carrying the token on load	On load the app makes a backend call bearing the token

`token-sanity.spec.ts` — "Product API sanity via extracted UI token (dev)" (`@product @api @sanity`)

ID	CASE	ASSERTS	TAGS
TOK-01	The UI transmits a well-formed token to the backend	Extracted token has the expected shape	`@critical`
TOK-02	The extracted token matches the bundle public token in config	UI token equals the public token in `config.json`	—
TOK-03	Every UI backend call sends the token in the body over HTTPS, never the URL	Token only in request body over HTTPS; never a query param	`@security`

Run

```
QA_TARGET_ENV=dev npx playwright test --project=product tests/product/api
```

Notes

- Never use `waitForLoadState('networkidle')`. The embedded map iframe keeps the network busy, so it never goes idle; rely on `domcontentloaded` or on `expect` auto-waiting instead.
- Environmental outages are represented as typed errors (in `tests/product/support/errors.ts`). The decision to skip versus fail is made at the spec edge, not inside the page object.